

Large Language Models for Serverless Function Generation

An Investigation on FaaS Performance

Xinghan Chen, Robert Cordingly, Ling-Hong Hung, Wes Lloyd
School of Engineering and Technology
University of Washington Tacoma



December 15, 2025
The 11th International Workshop on Serverless Computing (WOSC 2025)

BE BOUNDLESS



Outline

- Background and Motivation
 - Research Questions
 - Methodology
 - Results
 - Conclusions

Serverless Computing

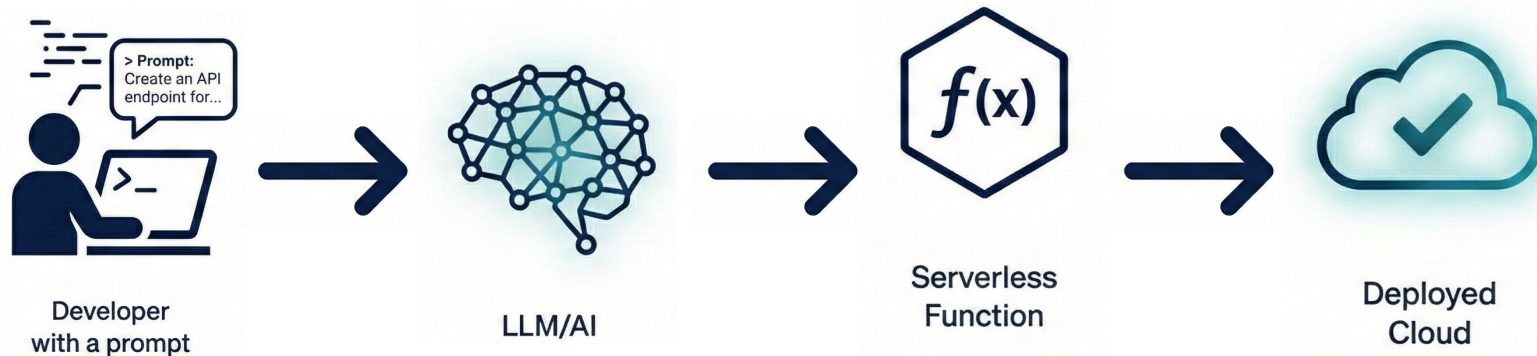
Function-as-a-Service

Serverless function-as-a-service (FaaS) platforms offer many desirable features:

- Rapid elastic scaling
- Scale to zero
- No infrastructure management
- Fine grained billing
- Fault tolerance
- No up front cost to deploy an application

The New Paradigm

Prompt, Generate, Deploy

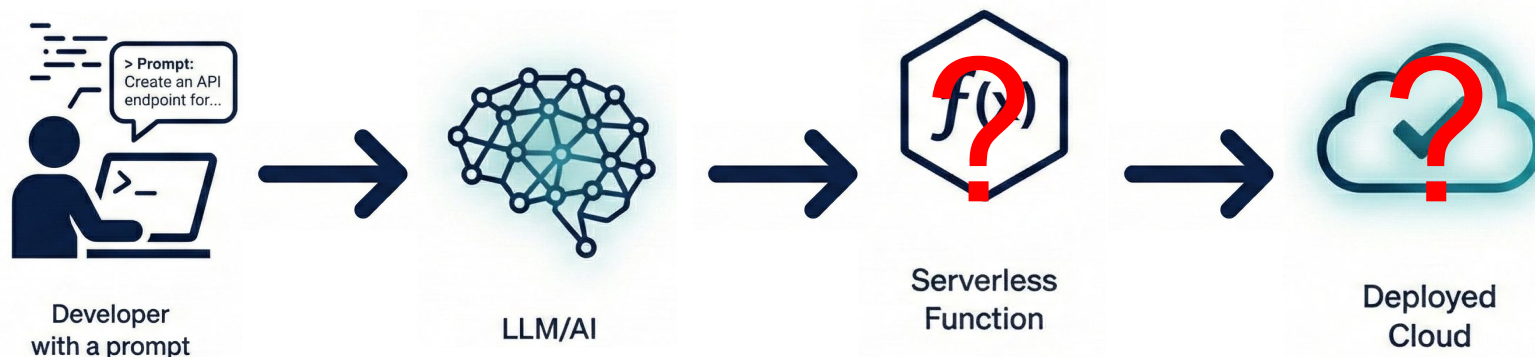


For coding tasks, Large Language Models (LLMs) can:

- > Generate function code, infrastructure and config with natural language
- > Archive faster deployment, cycles and higher developer productivity
- > Lower the barrier to entry to build complex cloud applications

But — What is the *REAL* Quality of this generated code?

The Hidden Challenge



Little prior work evaluates LLM-generated serverless code for FaaS functions, especially for:

- > Correctness: Does it actually work as intended?
- > Performance: Is it efficient, or silently slow?
- > Cost: Is it burning through the cloud budget?

Model Class / Size

Large: >100B Parameters

Impossible or super slow to run in local 1 GPU environment

Medium: >10B Parameters

Local runtime is possible but need high end hardware

Small: <10B Parameters

Can run on general available hardware

Outline

- Background and Motivation
-  Research Questions
- Methodology
- Results
- Conclusions

Research Questions

RQ-1: (Generated Code Quality)

What differences in code quality are observed when using smaller vs. larger LLMs to generate serverless function code for common algorithms, interview-style questions, and potentially unseen custom problems?

Research Questions

RQ-2: (Performance and Cost)

What runtime performance and cloud execution cost differences occur on serverless FaaS platforms when using smaller vs. larger LLMs to generate serverless function code for common algorithms, interview-style algorithms, and custom problems potentially unseen by the LLMs when deployed on a public FaaS platform with standard resource configurations?

Outline

- Background and Motivation
- Research Questions
-  Methodology
- Results
- Conclusions

Test Environment

FaaS Runtime:

AWS Lambda us-west-2 (Oregon)

CPU Model: Intel Xeon Platinum 8259CL

Memory size: 3008MB (3GB) with 2 vCPU cores

5GB ephemeral disk for I/O related tests

Workloads / Code Generation Prompts

**10 tasks &
16 total prompts**

- **Data analytics on CSVs**
- **Algorithmic / CPU-bound tasks**
 - **Graph kernel**
 - **Image processing**
 - **LeetCode-style problems**

Table 3: Summary of Code Generation Prompts

Prompt Name	Description	Token Count	Source
Health care 1	Healthcare payment records analysis from a CSV input	3,306	Original
Health care 2	State funding reduction calculation from a CSV input	3,244	Original
Sales Analysis	Daily sales income and order analysis	751	Original
Perfect number	Generation of perfect numbers up to n	382	Original
Prime number	Generation of prime numbers up to n	353	Original
Pagerank*	PageRank calculation on graph	367/352/261/254	SeBS [35]
Thumbnailer*	Image thumbnail generation	405/368/255/236	SeBS [35]
Minimal Cost Split	Minimum cost array splitting	308	LeetCode [22]
Car collision	Car fleet collision time computation	682	LeetCode [20]
Median of two array	Median calculation for sorted arrays	452	LeetCode [21]

Workloads / Code Generation Prompts

**10 tasks &
16 total prompts**

- **Data analytics on CSVs**
- **Algorithmic / CPU-bound tasks**
 - **Graph kernel**
 - **Image processing**
 - **LeetCode-style problems**

Table 3: Summary of Code Generation Prompts

Prompt Name	Description	Token Count	Source
Health care 1	Healthcare payment records analysis from a CSV input	3,306	Original
Health care 2	State funding reduction calculation from a CSV input	3,244	Original
Sales Analysis	Daily sales income and order analysis	751	Original
Perfect number	Generation of perfect numbers up to n	382	Original
Prime number	Generation of prime numbers up to n	353	Original
Pagerank*	PageRank calculation on graph	367/352/261/254	SeBS [35]
Thumbnailer*	Image thumbnail generation	405/368/255/236	SeBS [35]
Minimal Cost Split	Minimum cost array splitting	308	LeetCode [22]
Car collision	Car fleet collision time computation	682	LeetCode [20]
Median of two array	Median calculation for sorted arrays	452	LeetCode [21]

Workloads / Code Generation Prompts

**10 tasks &
16 total prompts**

- **Data analytics on CSVs**
- **Algorithmic / CPU-bound tasks**
 - **Graph kernel**
 - **Image processing**
 - **LeetCode-style problems**

Table 3: Summary of Code Generation Prompts

Prompt Name	Description	Token Count	Source
Health care 1	Healthcare payment records analysis from a CSV input	3,306	Original
Health care 2	State funding reduction calculation from a CSV input	3,244	Original
Sales Analysis	Daily sales income and order analysis	751	Original
Perfect number	Generation of perfect numbers up to n	382	Original
Prime number	Generation of prime numbers up to n	353	Original
Pagerank*	PageRank calculation on graph	367/352/261/254	SeBS [35]
Thumbnailer*	Image thumbnail generation	405/368/255/236	SeBS [35]
Minimal Cost Split	Minimum cost array splitting	308	LeetCode [22]
Car collision	Car fleet collision time computation	682	LeetCode [20]
Median of two array	Median calculation for sorted arrays	452	LeetCode [21]

Workloads / Code Generation Prompts

**10 tasks &
16 total prompts**

- **Data analytics on CSVs**
- **Algorithmic / CPU-bound tasks**
 - **Graph kernel**
 - **Image processing**
 - **LeetCode-style problems**

Table 3: Summary of Code Generation Prompts

Prompt Name	Description	Token Count	Source
Health care 1	Healthcare payment records analysis from a CSV input	3,306	Original
Health care 2	State funding reduction calculation from a CSV input	3,244	Original
Sales Analysis	Daily sales income and order analysis	751	Original
Perfect number	Generation of perfect numbers up to n	382	Original
Prime number	Generation of prime numbers up to n	353	Original
Pagerank*	PageRank calculation on graph	367/352/261/254	SeBS [35]
Thumbnailer*	Image thumbnail generation	405/368/255/236	SeBS [35]
Minimal Cost Split	Minimum cost array splitting	308	LeetCode [22]
Car collision	Car fleet collision time computation	682	LeetCode [20]
Median of two array	Median calculation for sorted arrays	452	LeetCode [21]

LLM Models

Class: small, medium, large models

Runtime: cloud vs. local deployment

Style: traditional next-token vs. reasoning-oriented models

Token: Bidirectional (input+output) word limit

Table 1: LLMs used for Serverless Function Code Generation

Vendor	Model	Class	Runtime	Size	Style	Tokens
OpenAI	gpt-4o [27]	Large	Cloud	~1.8T	traditional	128,000
OpenAI	gpt-4o-mini [27]	Small	Cloud	8B	traditional	128,000
OpenAI	o1-mini [28]	Small	Cloud	Small	reasoning	128,000
OpenAI	o1 [28]	Large	Cloud	Large	reasoning	200,000
OpenAI	o3-mini [29]	Small	Cloud	Small	reasoning	200,000
DeepSeek	deepseek-reasoner [12]	Large	Cloud	671B/32B	reasoning	131,072
DeepSeek	deepseek-r1:8b [12]	Small	Local	8B	reasoning	131,072
DeepSeek	deepseek-r1:32b [12]	Medium	Local	32B	reasoning	131,072
Alibaba	qwen2.5-coder:7b [1]	Small	Local	7B	traditional	131,072
Facebook	llama3.3:70b [25]	Medium	Local	70B	traditional	8,192
Xai	grok-3-beta [38]	Large	Cloud	314B	traditional	131,072
Xai	grok-3-mini-beta [38]	Small	Cloud	Small	traditional	131,072
Google	gemini-2.5-pro-exp-03-25 [16]	Large	Cloud	Large	traditional	1,000,000
Google	gemini-2.0-flash [16]	Large	Cloud	Large	traditional	1,000,000

LLM Models

Class: small, medium, large models

Runtime: cloud vs. local deployment

Style: traditional next-token vs. reasoning-oriented models

Token: Bidirectional (input+output) word limit

Table 1: LLMs used for Serverless Function Code Generation

Vendor	Model	Class	Runtime	Size	Style	Tokens
OpenAI	gpt-4o [27]	Large	Cloud	~1.8T	traditional	128,000
OpenAI	gpt-4o-mini [27]	Small	Cloud	8B	traditional	128,000
OpenAI	o1-mini [28]	Small	Cloud	Small	reasoning	128,000
OpenAI	o1 [28]	Large	Cloud	Large	reasoning	200,000
OpenAI	o3-mini [29]	Small	Cloud	Small	reasoning	200,000
DeepSeek	deepseek-reasoner [12]	Large	Cloud	671B/32B	reasoning	131,072
DeepSeek	deepseek-r1:8b [12]	Small	Local	8B	reasoning	131,072
DeepSeek	deepseek-r1:32b [12]	Medium	Local	32B	reasoning	131,072
Alibaba	qwen2.5-coder:7b [1]	Small	Local	7B	traditional	131,072
Facebook	llama3.3:70b [25]	Medium	Local	70B	traditional	8,192
Xai	grok-3-beta [38]	Large	Cloud	314B	traditional	131,072
Xai	grok-3-mini-beta [38]	Small	Cloud	Small	traditional	131,072
Google	gemini-2.5-pro-exp-03-25 [16]	Large	Cloud	Large	traditional	1,000,000
Google	gemini-2.0-flash [16]	Large	Cloud	Large	traditional	1,000,000

LLM Models

Class: small, medium, large models

Runtime: cloud vs. local deployment

Style: traditional next-token vs. reasoning-oriented models

Token: Bidirectional (input+output) word limit

Table 1: LLMs used for Serverless Function Code Generation

Vendor	Model	Class	Runtime	Size	Style	Tokens
OpenAI	gpt-4o [27]	Large	Cloud	~1.8T	traditional	128,000
OpenAI	gpt-4o-mini [27]	Small	Cloud	8B	traditional	128,000
OpenAI	o1-mini [28]	Small	Cloud	Small	reasoning	128,000
OpenAI	o1 [28]	Large	Cloud	Large	reasoning	200,000
OpenAI	o3-mini [29]	Small	Cloud	Small	reasoning	200,000
DeepSeek	deepseek-reasoner [12]	Large	Cloud	671B/32B	reasoning	131,072
DeepSeek	deepseek-r1:8b [12]	Small	Local	8B	reasoning	131,072
DeepSeek	deepseek-r1:32b [12]	Medium	Local	32B	reasoning	131,072
Alibaba	qwen2.5-coder:7b [1]	Small	Local	7B	traditional	131,072
Facebook	llama3.3:70b [25]	Medium	Local	70B	traditional	8,192
Xai	grok-3-beta [38]	Large	Cloud	314B	traditional	131,072
Xai	grok-3-mini-beta [38]	Small	Cloud	Small	traditional	131,072
Google	gemini-2.5-pro-exp-03-25 [16]	Large	Cloud	Large	traditional	1,000,000
Google	gemini-2.0-flash [16]	Large	Cloud	Large	traditional	1,000,000

LLM Models

Class: small, medium, large models

Runtime: cloud vs. local deployment

Style: traditional next-token vs. reasoning-oriented models

Token: Bidirectional (input+output) word limit

Table 1: LLMs used for Serverless Function Code Generation

Vendor	Model	Class	Runtime	Size	Style	Tokens
OpenAI	gpt-4o [27]	Large	Cloud	~1.8T	traditional	128,000
OpenAI	gpt-4o-mini [27]	Small	Cloud	8B	traditional	128,000
OpenAI	o1-mini [28]	Small	Cloud	Small	reasoning	128,000
OpenAI	o1 [28]	Large	Cloud	Large	reasoning	200,000
OpenAI	o3-mini [29]	Small	Cloud	Small	reasoning	200,000
DeepSeek	deepseek-reasoner [12]	Large	Cloud	671B/32B	reasoning	131,072
DeepSeek	deepseek-r1:8b [12]	Small	Local	8B	reasoning	131,072
DeepSeek	deepseek-r1:32b [12]	Medium	Local	32B	reasoning	131,072
Alibaba	qwen2.5-coder:7b [1]	Small	Local	7B	traditional	131,072
Facebook	llama3.3:70b [25]	Medium	Local	70B	traditional	8,192
Xai	grok-3-beta [38]	Large	Cloud	314B	traditional	131,072
Xai	grok-3-mini-beta [38]	Small	Cloud	Small	traditional	131,072
Google	gemini-2.5-pro-exp-03-25 [16]	Large	Cloud	Large	traditional	1,000,000
Google	gemini-2.0-flash [16]	Large	Cloud	Large	traditional	1,000,000

Using SAAF in a Function:

Using SAAF in a function is as simple as importing the framework and adding a couple lines of code. Attributes collected by SAAF will be appended onto the JSON response. For asynchronous functions, this data could be stored into a database, such as AWS S3, and retrieved after the function is finished.

Example Function:

```
from Inspector import *

def myFunction(request):
    # Initialize the Inspector and collect data.
    Inspector = Inspector()
    Inspector.inspectAll()

    # Add a "Hello World!" message.
    Inspector.addAttribute("message", "Hello " + request['name'] + "!")

    # Return attributes collected.
    return Inspector.finish()
```

Example Output JSON:

The attributes collect can be customized by changing which functions are called. For more detailed descriptions of each variable and the functions that collect them, please see the framework documentation for each language.

```
{
  "version": "0.3",
  "lang": "python",
  "cpuType": "Intel(R) Xeon(R) Processor @ 2.50GHz",
  "cpuModel": "63",
  "vmuptime": "1551727835",
  "uuid": "6241c181-7808-48e2-8736-997dc1a93164",
  "volid": "f113C6A",
  "platform": "AWS Lambda",
  "newContainer": 1,
  "cpuUserDelta": "98a",
  "cpuKernelDelta": "9a",
  "cpuIdleDelta": "585",
  "cpuUserDelta": "82428",
  "cpuKernelDelta": "226",
  "cpuIdleDelta": "a",
  "cpuSoftirqDelta": "y",
  "requestsDelta": "1594",
  "frameworkRuntime": "15,72",
  "message": "Hello Fred Smith",
  "runtime": "38.94"
}
```

Attributes Collected by Each Function

The amount of data collected is determined by which functions are called. If some attributes are not needed, then some functions may not need to be called. If you would like to collect every attribute, the `inspectAll()` method will run all methods.

Core Attributes	
Field	Description
version	The version of the SAAF Framework.
lang	The language of the function.
runtime	The server-side runtime when the function is initiated until <code>Inspector.finish()</code> is called.
startTime	The Unix Epoch that the Inspector was initialized in ms.

inspectContainer()	
Field	Description
uuid	A unique identifier assigned to a container if one does not already exist.
newContainer	Whether a container is new (no assigned uuid) or if it has been used before.
vmuptime	Time when the host booted in seconds since January 1, 1970 (Unix epoch).

inspectCPU()	
Field	Description
cpuType	The model name of the CPU.
cpuModel	The model name of the CPU.

Supporting Tools - SAAF

We utilize the Serverless Application Analytics Framework to collect metrics from serverless functions.

Metrics including CPU time accounting metrics (CPU User, CPU Kernel, CPU Idle), runtime, latency, and more

SAAF Gathers data during function execution provides inputs for training performance models.

SAAF and our other tools are available here:

<https://github.com/wlloyduw/SAAF>

Outline

- Background and Motivation
- Research Questions
- Methodology
-  Results
- Conclusions

Research Question 1

**LLM-Generated
Serverless Code Quality**

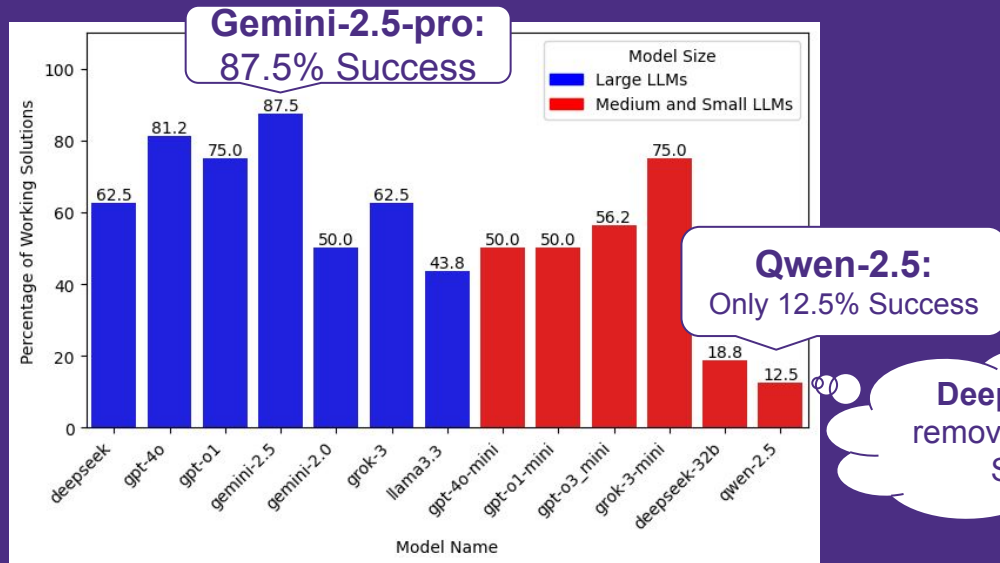
**How does LLM size
affect code quality for
serverless functions?**

Small vs. Large models:

- common algorithms
- interview-style questions
- custom problems

Functional Correctness Is a Minefield

Larger models succeed more often, but smaller, locally-runnable models struggle significantly.



Large models: higher success rate (66.07%).

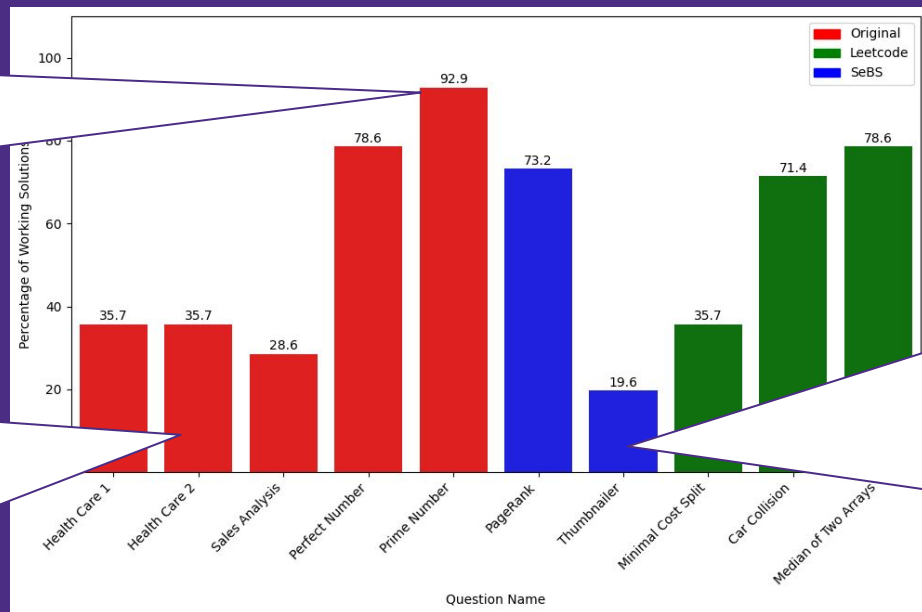
Small/Medium: more failures w/success rate only (29.66%).

Deepseek-8b is removed due to 0% Success

Correctness Varies Dramatically by Task Complexity

LLMs performed well on well-defined algorithmic tasks like Prime Number Generator(92.9% success across all models

However, they struggled with more complex, IO bound tasks with specific data schemas, like our original tasks Sales Analusis(28.6%) and Health Care 1/2 (35.7%)



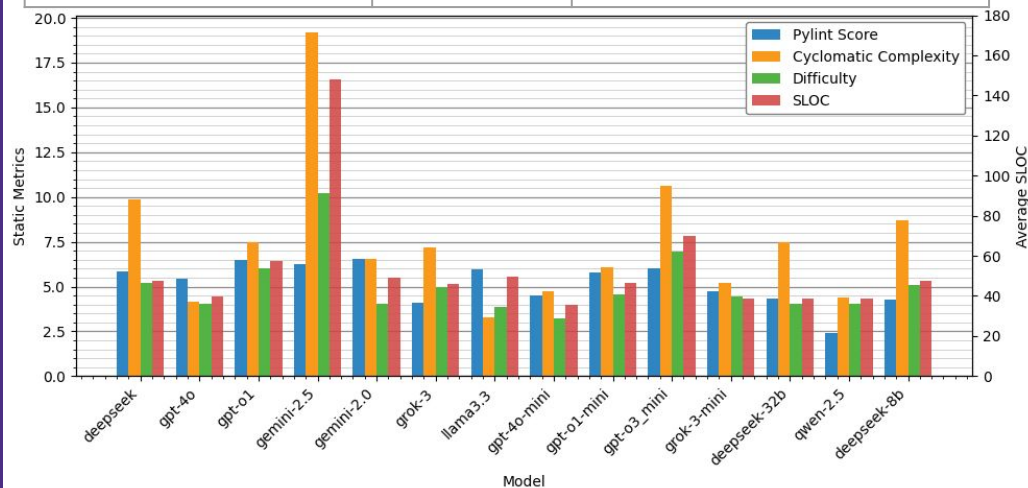
Percentage of Functionally Correctness function by Prompt

Even tasks from standard benchmarks like Thumbnailer (19.6%) from SeBS, highlighting challenges with library usage and image processing logic

Static Analysis

- Tools: Pylint, Radon (complexity, maintainability).
- Larger models produce simpler, more idiomatic code with higher lint scores (Pylint: 5.8).
- Smaller models produce code with lower scores and sometimes more convoluted logic (Pylint: 4.58).

	Large	Medium/Small
Score	5.8	4.58
Works	10.57	6.00
Complexity	8.25	6.75
Difficulty	5.48	4.63
SLOC	62.45	45.03



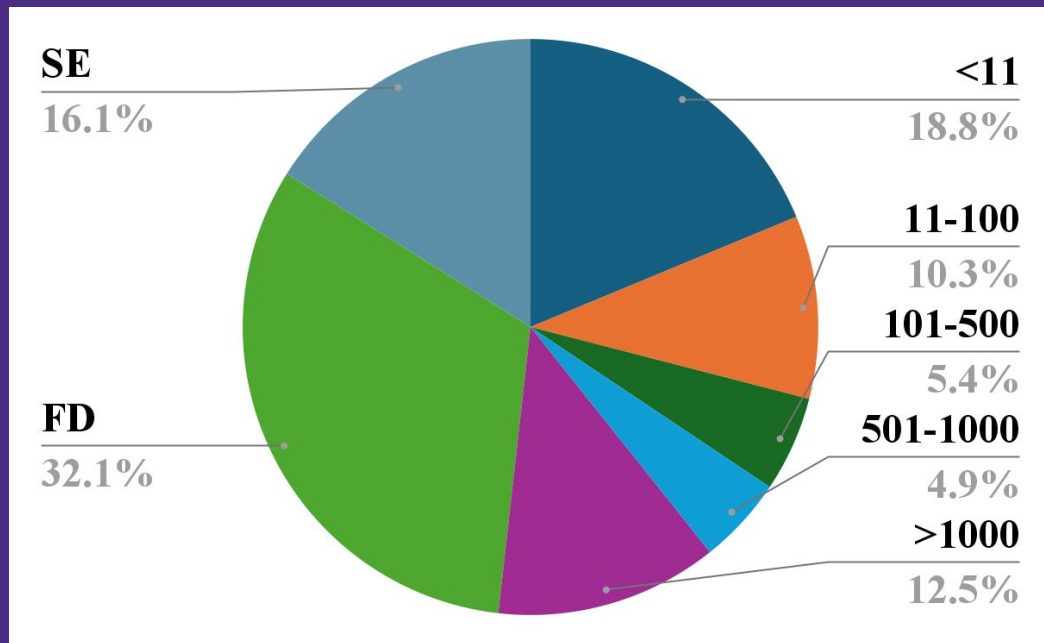
Research Question 2

LLM-Generated
Serverless Code Runtime

**How does LLM size
impact runtime
performance and
cloud execution cost
on FaaS?**

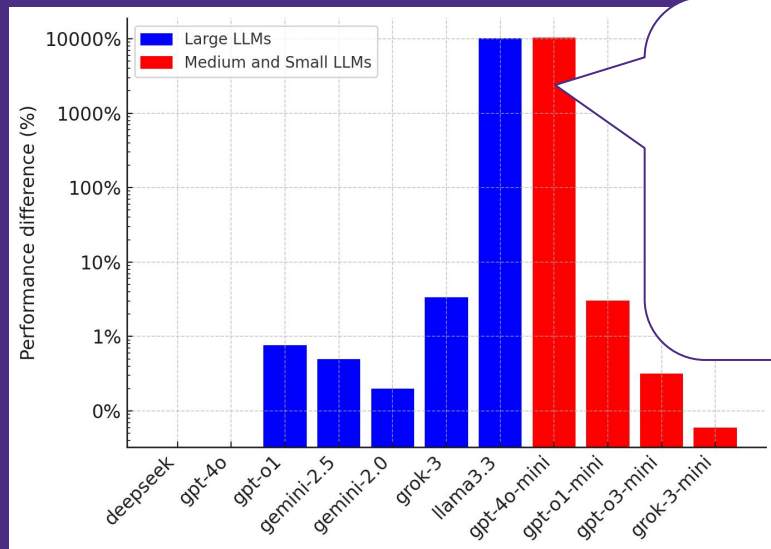
Approximately half of our generated functions work

- 51.8%: were functionally correct
- 16.1%: Syntax Error (SE)
- 32.1%: Functional Defect (FD)



Correct Code Can Be Catastrophically Slow

Some LLMs generate functionally correct code that is over 100 times slower than a human-written baseline.

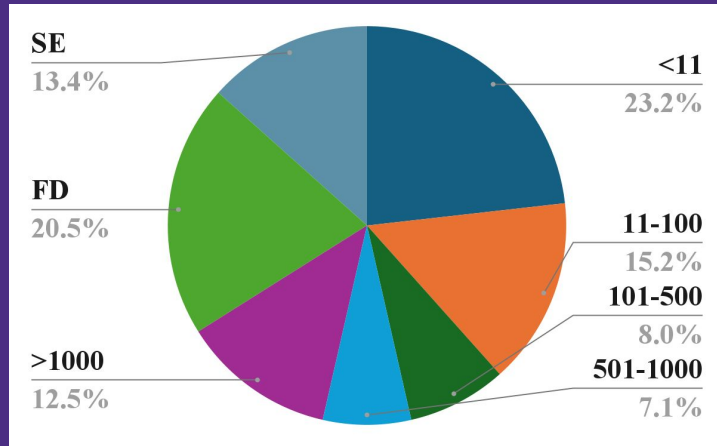


>100x

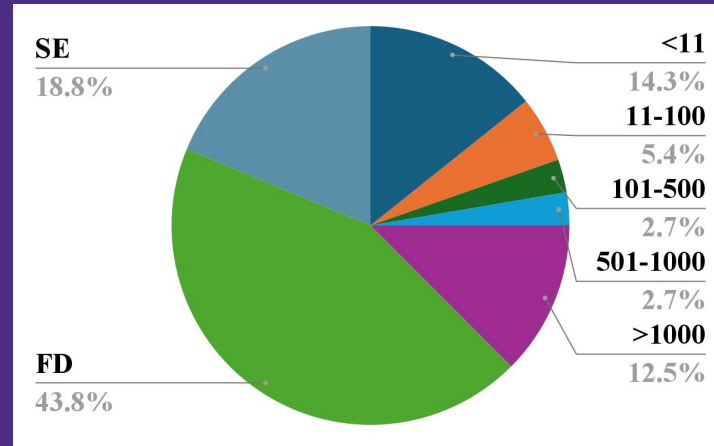
Slower than baseline

Large vs. Medium / Small models

Larger models tended to generate faster (43/112 acceptable performance) FaaS functions than smaller models (22/112).



Large Models

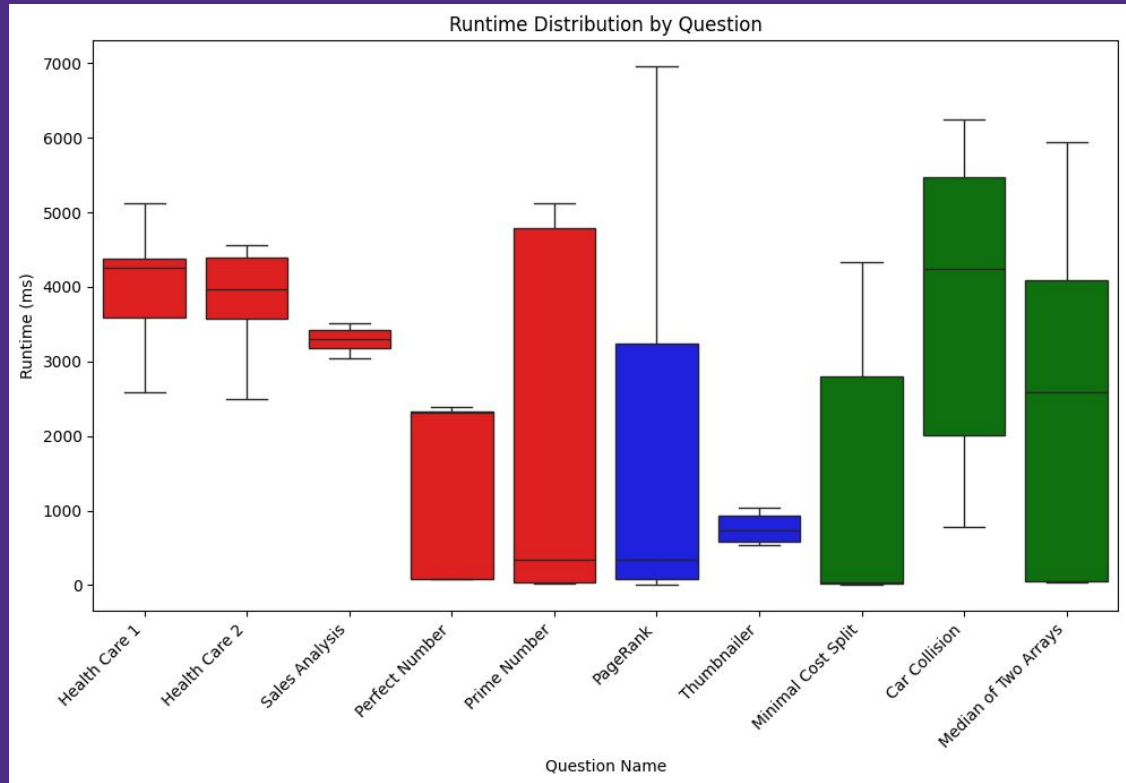


Medium/Small Models

*Acceptable:
within 100%
more runtime
compare to
baseline

Difference of Tasks

Differences between models are smaller (0.95x) for I/O-bound tasks (Health Care 1/2, Sales Analysis) compare to CPU-bound tasks (6.39x) like Prime Number or PageRank.

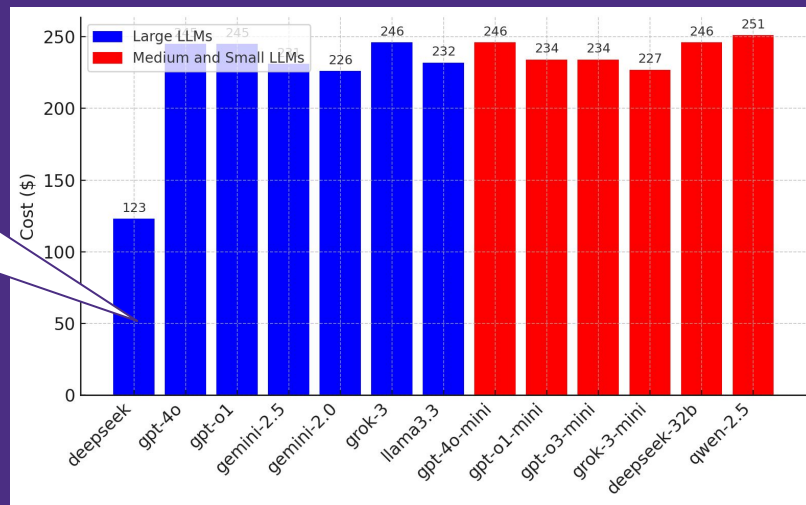


Low performance = Wasted Money

Poor performance directly translates to higher cloud costs

The right model led to a
59.4% cost savings for the
same task.

Faster code from larger
LLMs can lower overall
Lambda execution costs,
even though the models
themselves can have higher
inferencing costs to run.



Cost of 1 Million Invocations (Prime Number)

Outline

- Background and Motivation
- Research Questions
- Methodology
- Results

 Conclusions

Conclusion

RQ-1 – Code Quality

- Large cloud LLMs are more likely to generate functionally correct serverless functions.
- Their code also scores better on style and maintainability metrics.
- Smaller/local models work on simpler tasks, but fail more often on complex or unfamiliar workloads.

Conclusion

RQ-2 – Performance & Cost

- For CPU-heavy tasks, code from large models tends to run faster and can lower overall Lambda execution costs.
- For I/O-bound tasks, runtime gaps are smaller, so the benefit of larger models is less pronounced.
- Smaller/local models may still be attractive when data locality or strict cost controls matter, but they can increase debugging effort and runtime cost.

Thank You!

Future work——

Agentic Code Optimization for Serverless Computing

- > Runtime and cost: Performance, Efficiency, and Scalability**
- > Robusty: Reliability, Availability, and Fault Tolerance**
- > Safety: Security, Compliance, and Data Integrity**
- > Long-term code health: Maintainability, Modularity, and Extensibility**

Code generation

Large Language Model(LLM)
based development

For coding tasks, Large Language Models (LLMs) can:

- Generate code from natural language prompts
- Speed up development
- Lower entry barriers

Little prior work evaluates LLM-generated serverless code for FaaS functions, especially for:

- Correctness
- Impact of model size and quality
- Runtime in the cloud

Functional Correctness

Measure: does output match expected results on test cases?

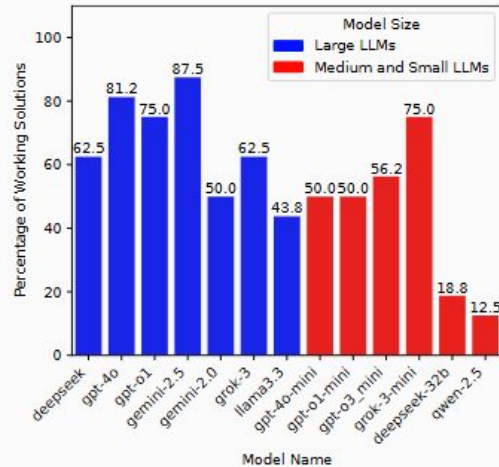


Figure 1: % of Functionally Correct Functions by LLM

Large models:
high success rate
(66.07%).

Small/Medium:
more failures w/
success rate only
(29.66%).

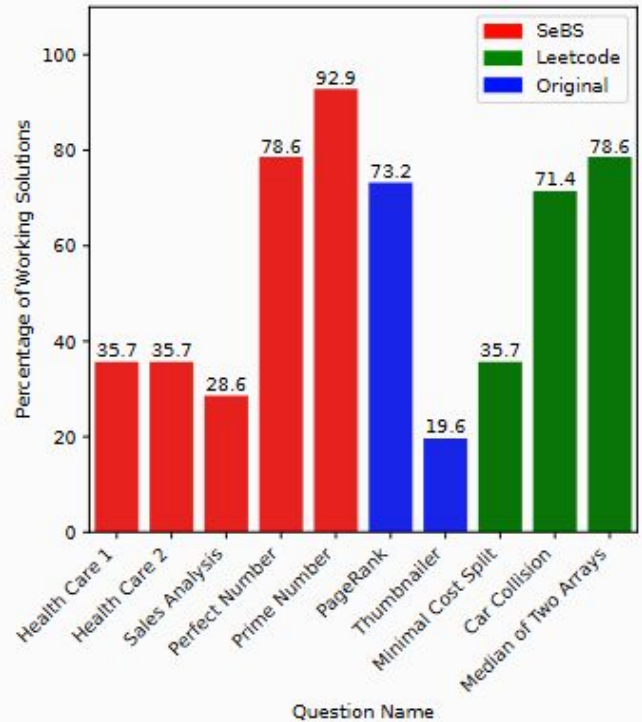


Figure 2: % of Functionally Correct Functions by Prompt

Runtime Difference

Large models produced (43/112) functions with acceptable performance results. Middle and small model only produced (22/112) functions for all of our prompts.

*Acceptable: within 100% more runtime compare to baseline

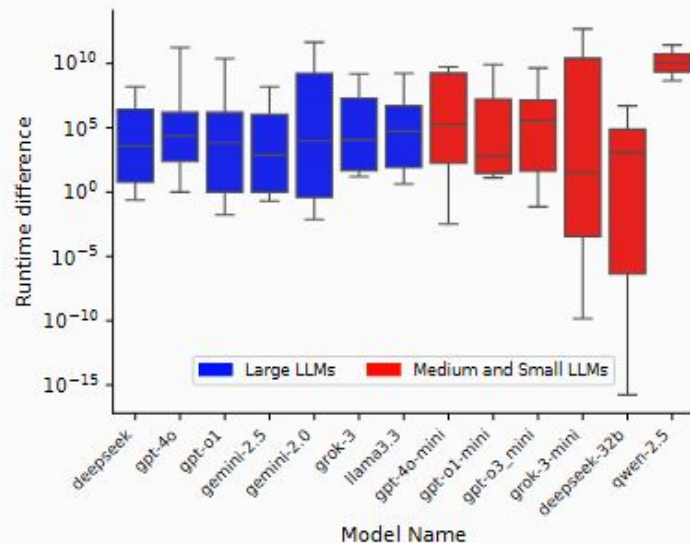


Figure 5: Runtime % Difference by LLM vs. Reference (log scale)

Cost

Pay more on code generation,
less on runtime.

